Andy, Yu-Guang Chen

EE6094 CAD for VLSI Design

Programming Assignment 2: k-input LUT Mapping

(Due: 23:59:59, 2026/04/23)

Introduction

In modern FPGA (Field-Programmable Gate Array) design flows, Technology Mapping is a key step that bridges the gap between high-level logic synthesis and physical implementation. The goal is to transform a technology-independent logic network into a netlist composed of k-bit input Look-Up Tables (LUTs), which are the fundamental building blocks of FPGAs. This assignment focuses on implementing a LUT Mapping algorithm by modeling logic circuits as And-Inverter Graphs (AIGs) and partitioning them into valid k-input subgraphs.

In this assignment, you will:

1. Understand BLIF structure: Parse and generate the Berkeley Logic Interchange Format (BLIF), the industry-standard exchange format for logic synthesis.
2. Represent logic circuits using AIGs: Convert a standard logic description into a directed acyclic graph (DAG) consisting only of two-input AND nodes and inverters to simplify logic manipulation.
3. Implement k-bit input LUT Mapping: Develop algorithms to identify feasible cuts within the AIG and cover the network with the minimum number of LUTs while following the k-input constraint.

Background

FPGAs are primarily composed of Look-Up Tables (LUTs). Unlike traditional standard cell libraries that use fixed logic gates (like NAND or OR), a k-bit LUT is a flexible logic element that can implement any Boolean function as long as the number of inputs is no more than k.

To process these logic functions, early synthesis tools often used Binary Decision Diagrams (BDDs). However, while BDDs provide a canonical representation of logic, they often suffer from "memory explosion" when dealing with complex circuits or specific functions like multipliers. Modern tools like ABC have shifted to And-Inverter Graphs (AIGs). An AIG is a specialized DAG where each node has exactly two inputs representing a logical AND, and edges can be "inverted" to represent NOT gates. This representation is much more scalable and allows for efficient structural analysis. The mapping process involves finding a set of "cuts" (subgraphs with a limited number of inputs) that cover the entire AIG. The quality of the mapping is measured by the total number of LUTs and the maximum delay of the design.

Problem Definition

Given a logic network in BLIF format and a specified constraint k, your task is to implement a mapping tool that outputs a new BLIF file consisting only of k-input LUTs.

There are 3 steps to follow:

- Step1. **Build the AIG Representation**: Parse the input BLIF file and construct a corresponding And-Inverter Graph (AIG). Complex logic gates (represented by .names in BLIF) must be correctly decomposed into 2-input AND nodes and inverters.
- Step 2. **Generate and Select Cuts**: Enumerate all possible cuts for each node in the AIG, ensuring no cut exceeds k inputs. Then, select an optimal set of cuts to cover the entire circuit.
- Step 3. **Generate LUT-based BLIF Output**: Translate the selected cuts into k-input LUTs. Each LUT must be represented as a ". names" block in the output BLIF file, ensuring that the final circuit preserves the original logic functionality.

Input file format

The input file follows the Berkeley Logic Interchange Format (BLIF). To focus on the core Mapping algorithm, the test cases provided in this assignment will only use a specific subset of BLIF directives. Your parser should be able to process the following blocks:

- .model: Defines the name of the logic network.
- .inputs: Lists the primary input signals of the circuit.
- .outputs: Lists the primary output signals of the circuit.
- .names: Describes a logic node and its corresponding Boolean function.
- .end: Marks the end of the model.

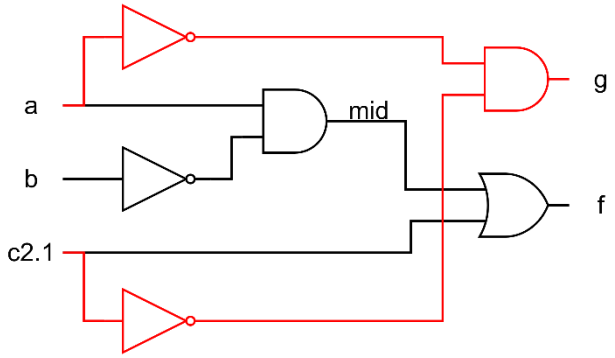
Important Parsing Notes:

- Special Characters: Signal names (inputs, outputs, and internal nets) as well as the model name may contain the dot symbol (.). For example, "node.1" or "v10.2" are valid names. Do not confuse these with BLIF directives (like .names).
- Line Continuation: If a line is too long, it may be split using a backslash (\) at the end of the line. Your program must treat the next line as a continuation of the current one.
- Robustness: Your program should handle extra spaces, TABs, and empty lines. Comments start with the hash symbol (#) and should be ignored.

```
.model example.501
.inputs a b c2.1
.outputs f g

.names a \
b mid
10 1

.names mid c2.1 f
1- 1
-1 1

.names a c2.1 g
00 1

.end
```

←backslash
Seen as .names a b mid

f = mid + c2.1
here '-' means don't care



mid = a * !b
f = mid + c2.1
g = !a * !c2.1

Fig 1. An example of the BLIF file and its corresponding description

The .names directive defines a logic unit and its Truth Table (represented in Sum of Products form).

- The Declaration Line: The .names keyword is followed by a list of signals. The last signal is the output of this logic block; all signals before it are the inputs.
- The Logic Rows (Truth Table): Each line following the declaration describes a condition (Product Term) that makes the output 1. Will use 1 (true), 0 (inverted/NOT), or - (don't care).

Output file format

Your program must output the mapping results in a valid BLIF format. A valid mapping must satisfy the following constraints:

- k-input Constraint: For every .names block in your output file, the number of input signals must not exceed k (where $2 \leqq k \leqq 10$ is the input argument).
- I/O Matching: The .model name, .inputs list, and .outputs list must be identical to the input file. Do not rename or reorder primary I/Os.
- Functional Correctness: For any input pattern, the logic values at the primary outputs must match the original circuit exactly.
- No Floating Wires: Every LUT input must be driven by a Primary Input or another LUT output.

- No Multi-driven Nets: Each signal name can only be the output of **one** .names block.
- No Combinational Loops: The circuit must remain a Directed Acyclic Graph (DAG).

Quality

The quality of your mapping result will be evaluated based on the total implementation cost. A smaller cost indicates a more efficient mapping. Note that the quality points will be 0 if your result does not meet all constraints in Output file format. The cost is calculated using the following formula:

$$Cost = Level * |LUTs|$$

- Level: The maximum number of LUTs on the longest path from any primary input to any primary output.
- |LUTs|: The total count of .names blocks in your output BLIF file.

Requirement

1. You have to write this program in C or C++. No open-source codes are allowed to be used. (i.e., you MUST implement the tool by yourself, except for the use of ABC Tool in Bonus). You can use any data structure to realize your program.
2. The run time of your program is limited to at most 5 minutes per testcase.
3. Your submission must consist of **two files**:
   - A .tar archive named **StudID_PA2.tar** (e.g., 9862534_PA2.tar). This archive should contain your source code and a Makefile. The directory structure inside the archive should follow the format below:

```
9862534_PA2/          (Project Folder)
|-- 9862534_PA2.cpp   (Main source file)
|-- Makefile          (Compilation and running script)
|-- inc/              (Header files, if needed)
|     |-- file1.h
|     |-- file2.h
|-- src/              (Source files, if needed)
|     |-- file1.cpp
|     |-- file2.cpp
```

   - A PDF report named **StudID_PA2_report.pdf** (e.g., 9862534_PA2_report.pdf). This file should contain your report.
   - Note:
     (1) Incorrect naming results in a 10% penalty.
     (2) Only .pdf reports are accepted; .doc or .docx will not be graded.
     (3) No duplicate file extensions (e.g., 9862534_PA2.cpp.cpp is incorrect).
     (4) Including the testcase files in a 10% penalty.

4. Compilation & Execution Command
   - Compile: This command compiles your code and generates the executable file.

     $make all
   - Execute: This command runs the executable with the provided input file path, output file path and the constraint k.

     $make run input=<input.blif> output=<output.blif> k=<number, 2~10>
   - Clean: This command will automatically remove all the objects and executable file generated by make all.

     $make clean
   - We will evaluate your code based on the Makefile you submit. Ensure your Makefile compiles and runs correctly, otherwise you will lose points for both correctness and quality.
5. Report Guidelines
   - We don't restrict the report format and length. In your report, you have to at least include:
     (1) How to compile and execute your program (You can use screenshot to explain)
     (2) Completion status of the assignment (If you complete all requirements, just specify all)
     (3) Algorithms and data structures used
     (4) The hardness of this assignment and how you overcome it
     (5) Any suggestions about this programming assignment?

Bonus

Bonus details will be announced in a separate document.

Grading

The grading is as follows:

1. Correctness of your results: 35%
   - Parse BLIF: 5%
   - Convert to And-Inverter Graph (AIG): 10%
   - Mapping to k-bit FPGA LUTs: 20%
2. Quality of your results 10%
   - 3 public + 2 hidden test cases: 2%/case
3. Readability of your code: 10%
4. The report: 20
5. Demo session: 25%
6. (Bonus) ABC Tool Use 10%
   - Use Command to mapping LUTs: 5%
   - Verify and visualize your answer: 5%

Please submit your assignment on time. Otherwise, the penalty rule will apply:

- Within 24hrs delay: 20% off
- Within 48hrs delay: 40% off
- More than 48hrs: 0 point

Contact

For all questions about PA2, please send Email to TA 吳育丞 mailto:mineeric05@gmail.comwith the title [CAD PA2].